



ARRAY COM FUNÇÕES NATIVAS

ESTRUTURA DE DADOS

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



PARA SOBREVIVER AO JAVASCRIPT

Non-zero value



null



0



undefined



ADICIONANDO E REMOVENDO ITENS

Essas funções alteram diretamente o array original

MÉTODOS push() E unshift()



Aula 09 - Array Funções Nativas - app.js

```
12  const fila = ['Beto', 'Carlos'];  
13  
14  fila.push('Daniel');    // ['Beto', 'Carlos', 'Daniel']  
15  fila.unshift('Ana');    // ['Ana', 'Beto', 'Carlos', 'Daniel']  
16  
17  const primeiro = fila.shift(); // Remove 'Ana'. Sobra: ['Beto', 'Carlos', 'Daniel']  
18  const ultimo = fila.pop();    // Remove 'Daniel'. Sobra: ['Beto', 'Carlos']
```

EXTRAINDO, CORTANDO E JUNTANDO

MÉTODO slice()



Aula 09 - Array Funções Nativas - app.js

```
30  const letras = ['A', 'B', 'C', 'D', 'E'];  
31  const meio = letras.slice(1, 4); // Pega do índice 1 até o 3  
32  
33  console.log(meio); // ['B', 'C', 'D']  
34  console.log(letras); // ['A', 'B', 'C', 'D', 'E'] (Intacto)
```

MÉTODO splice()



Aula 09 - Array Funções Nativas - app.js

```
41  const meses = ['Jan', 'Mar', 'Abr'];
42  meses.splice(1, 0, 'Fev'); // No índice 1, remove 0 itens, insere 'Fev'
43  // meses agora é ['Jan', 'Fev', 'Mar', 'Abr']
44
45  meses.splice(2, 1); // No índice 2, remove 1 item ('Mar')
46  // meses agora é ['Jan', 'Fev', 'Abr']
```

MÉTODO concat()



Aula 09 - Array Funções Nativas - app.js

```
54  const partes = ['Olá', 'Mundo'];  
55  console.log(partes.join(' ')); // "Olá Mundo"  
56  
57  const array1 = [1, 2];  
58  const array2 = [3, 4];  
59  console.log(array1.concat(array2)); // [1, 2, 3, 4]
```

TRANSFORMAÇÕES E FILTRAGENS

MÉTODO map()

● ● ● Aula 09 - Array Funções Nativas - app.js

```
76  const numeros = [1, 2, 3];  
77  const dobrados = numeros.map(n => n * 2);  
78  
79  console.log(dobrados); // [2, 4, 6]
```

MÉTODO filter()



Aula 09 - Array Funções Nativas - app.js

```
86  const idades = [15, 22, 18, 30, 12];  
87  const adultos = idades.filter(idade => idade >= 18);  
88  
89  console.log(adultos); // [22, 18, 30]
```

BUSCA E VERIFICAÇÕES

MÉTODO find()



Aula 09 - Array Funções Nativas - app.js

```
101  const usuarios = [  
102    { id: 1, nome: 'Ana' },  
103    { id: 2, nome: 'Beto' }  
104  ];  
105  
106  const usuario = usuarios.find(u => u.id === 2);  
107  console.log(usuario); // { id: 2, nome: 'Beto' }
```

MÉTODO `includes()`

   Aula 09 - Array Funções Nativas - app.js

```
113  const frutas = ['banana', 'maçã', 'uva'];  
114  console.log(frutas.includes('maçã')); // true
```

MÉTODO every()



Aula 09 - Array Funções Nativas - app.js

```
116  /**
117   * .every()
118   * Pense no .every() como um inspetor rigoroso. Ele testa todos os elementos
119   * do array contra uma condição. Se apenas um elemento falhar no teste, ele retorna false.
120   * Ele só retorna true se absolutamente todos os itens passarem.
121   * É excelente para validações em formulários ou checagens de permissão.
122   */
123  const convidados = [22, 25, 19, 30];
124
125  // Verifica se TODOS são maiores ou iguais a 18
126  const todosMaiores = convidados.every(idade => idade >= 18);
127
128  console.log(todosMaiores); // true (pois não há ninguém menor de 18)
```

MÉTODO indexOf()



Aula 09 - Array Funções Nativas - app.js

```
130  /**
131   * .indexOf()
132   * O que ele retorna: Se encontrar o item, retorna a posição (índice) dele, começando do 0.
133   * Se não encontrar: Ele sempre retorna -1. (Isso é muito útil para fazer validações).
134   * Imutável: Ele não altera o array original.
135   * Busca Exata: Ele usa igualdade estrita (===), ou seja, o valor e o tipo precisam ser
136   * exatamente iguais (o número 1 é diferente da string '1').
137   * O .indexOf() é excelente para arrays simples (strings, números, booleanos). No entanto,
138   * ele não funciona para buscar Objetos dentro de Arrays.
139   */
140  const crachasBloqueados = ['A123', 'B456', 'X999', 'Z001'];
141  const crachaPassado = 'X999';
142
143  const posicaoBloqueio = crachasBloqueados.indexOf(crachaPassado);
144
145  if (posicaoBloqueio !== -1) {
146    console.log(`Acesso Negado! Crachá encontrado na posição ${posicaoBloqueio} da lista negra.`);
147  } else {
148    console.log('Acesso Liberado.');
```

ACUMULAÇÃO E REDUÇÃO

MÉTODO reduce()



Aula 09 - Array Funções Nativas - app.js

```
124  const carrinho = [10.50, 20.00, 5.00];
125  const total = carrinho.reduce((acumulador, valorAtual) => {
126    return acumulador + valorAtual;
127  }, 0); // 0 é o valor inicial do acumulador
128
129  console.log(total); // 35.50
```

ITERAÇÃO SIMPLES

MÉTODO `forEach()`



Aula 09 - Array Funções Nativas - app.js

```
140 const nomes = ['Alice', 'Bob'];  
141 nomes.forEach(nome => console.log(`Olá, ${nome}!`));
```

ORDENAÇÃO DE ARRAYS

MÉTODO sort()



Aula 09 - Array Funções Nativas - app.js

```
184  /**
185   * .sort()
186   * O .sort() é usado para ordenar os itens de um array.
187   * Atenção a dois detalhes cruciais:
188   * - Ele altera o array original (é mutável).
189   * - Por padrão, ele converte os elementos em Strings e os ordena por ordem
190   *   alfabética (baseado na tabela Unicode).
191   * Por causa desse comportamento padrão, ordenar números diretamente sem uma
192   * função de apoio gera bugs bizarros
193   * (ex: o número 10 vem antes do 2, porque a letra "1" vem antes da letra "2").
194   */
195
196  const cesta = ['Banana', 'Abacaxi', 'Laranja'];
197  cesta.sort();
198
199  console.log(cesta); // ['Abacaxi', 'Banana', 'Laranja']
```

EXERCÍCIOS

EXERCÍCIO 1

- O RH precisa de uma lista apenas com os desenvolvedores de nível "Senior" para um novo projeto. Use uma função para retornar um novo array contendo apenas os funcionários com o cargo de "Senior".

● ● ● Aula 09 - Array Funções Nativas - Exer01.js

```
1  const funcionarios = [  
2    { nome: 'João', cargo: 'Junior' },  
3    { nome: 'Ana', cargo: 'Senior' },  
4    { nome: 'Carlos', cargo: 'Pleno' },  
5    { nome: 'Beatriz', cargo: 'Senior' }  
6  ];
```

EXERCÍCIO 2

- A empresa bateu a meta e todos os funcionários vão receber um bônus de 10% em cima do salário atual. Crie um novo array com os salários reajustados (multiplicados por 1.10).

 Aula 09 - Array Funções Nativas - Exer02.js

```
1  const salarios = [3000, 4500, 6000, 8500];
```

EXERCÍCIO 3

- O gerente de vendas quer saber o faturamento total do dia. Reduza o array de valores a um único número que represente a soma de todas as vendas.



Aula 09 - Array Funções Nativas - Exer03.js

```
1  const vendasDoDia = [150.50, 400.00, 20.00, 89.90];
```

EXERCÍCIO 4

- Um diretor ligou reclamando do chamado de urgência número 1043. O suporte precisa encontrar os detalhes desse chamado específico. Encontre e retorne o objeto exato correspondente ao id: 1043.



Aula 09 - Array Funções Nativas - Exer04.js

```
1  const chamados = [  
2    { id: 1041, status: 'Aberto', urgencia: 'Baixa' },  
3    { id: 1042, status: 'Fechado', urgencia: 'Média' },  
4    { id: 1043, status: 'Aberto', urgencia: 'Alta' }  
5  ];
```

EXERCÍCIO 5

- O setor de compliance precisa garantir que todos os funcionários da lista completaram o treinamento de segurança da informação. Verifique se todos os itens possuem a propriedade `treinamentoConcluido` como `true`. O retorno deve ser um booleano.



Aula 09 - Array Funções Nativas - Exer05.js

```
1  const equipe = [  
2    { nome: 'Marcos', treinamentoConcluido: true },  
3    { nome: 'Fernanda', treinamentoConcluido: true },  
4    { nome: 'Roberto', treinamentoConcluido: false }  
5  ];
```

EXERCÍCIO 6

- O sistema do Call Center funciona como uma fila contínua. Entrou um cliente novo na fila e o primeiro cliente foi atendido. Adicione o cliente "Zélia" ao final da fila e remova o primeiro cliente da fila. Imprima a nova fila.



Aula 09 - Array Funções Nativas - Exer06.js

```
1  const filaAtendimento = ['Augusto', 'Bruna', 'Camila'];
```

EXERCÍCIO 7

- O marketing tem uma lista de leads classificados por pontuação. Eles querem apenas os 3 primeiros para mandar um brinde, mas sem modificar a lista original. Extraia uma cópia dos 3 primeiros elementos do array.



Aula 09 - Array Funções Nativas - Exer07.js

```
1  const leadsRankeados = ['Lead A', 'Lead B', 'Lead C', 'Lead D', 'Lead E'];
```

EXERCÍCIO 8

- Na escala de plantão, "Daniel" ficou doente e "Diego" vai cobrir o turno dele exatamente na mesma posição da lista. Usando apenas uma função, remova "Daniel" e insira "Diego" no mesmo índice.



Aula 09 - Array Funções Nativas - Exer08.js

```
1  const escala = ['Carlos', 'Daniel', 'Eduardo', 'Felipe'];
```

EXERCÍCIO 9

- O armazém precisa listar os produtos do menor para o maior nível de estoque para saber o que comprar primeiro. Ordene os produtos de forma crescente baseando-se na propriedade quantidade.

 Aula 09 - Array Funções Nativas - Exer09.js

```
1  const estoque = [  
2    { produto: 'Teclado', quantidade: 45 },  
3    { produto: 'Mouse', quantidade: 12 },  
4    { produto: 'Monitor', quantidade: 30 }  
5  ];
```

EXERCÍCIO 10

- O CEO quer saber o custo total apenas dos softwares que estão com status "Ativo". Encadeie métodos (ex: filtre primeiro, depois some os valores) para descobrir o valor total gasto com licenças ativas.



Aula 09 - Array Funções Nativas - Exer10.js

```
1  const licencas = [  
2    { software: 'Pacote Office', custo: 1200, ativo: true },  
3    { software: 'Photoshop', custo: 800, ativo: false },  
4    { software: 'Antivírus', custo: 300, ativo: true },  
5    { software: 'CRM', custo: 2500, ativo: true }  
6  ];
```

OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: Google Classroom
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva@fatecjahu.edu.br

